2 Data transmission

In this chapter you will learn about:

- ★ types and methods of data transmission
  - how data is broken up into data packets before transmission
  - the structure of data packets (header, payload and trailer)
  - packet switching (including the role of the router in the process)
  - methods of data transmission (serial, parallel, simplex, half-duplex and full-duplex)
  - the Universal Serial Bus (USB)
- ★ methods of error detection
  - why error checking methods are needed
  - error checking methods following data transmission:
    - parity checks
    - checksum
    - echo check
  - use of check digits to detect data entry errors
  - use of automatic repeat requests (ARQs) to detect errors
- ★ encryption
  - the need for and the purpose of encryption
  - symmetric and asymmetric encryption
  - use of public and private keys.

Data is frequently transferred from one device to another. The two devices could be in the same building or thousands of kilometres away. Irrespective of the distance travelled, the transmission of data needs to be considered with respect to:

- how the data is transmitted
- how can errors following transmission be detected and can the data be recovered
- the role of encryption to make sure data that falls into the wrong hands can't be used

It is also important to consider ways of checking for errors in data once it has been entered into a computer.

2.1 Types and methods of data transmission

2.1.1 Data packets

Data packets are usually referred to simply as 'packets'

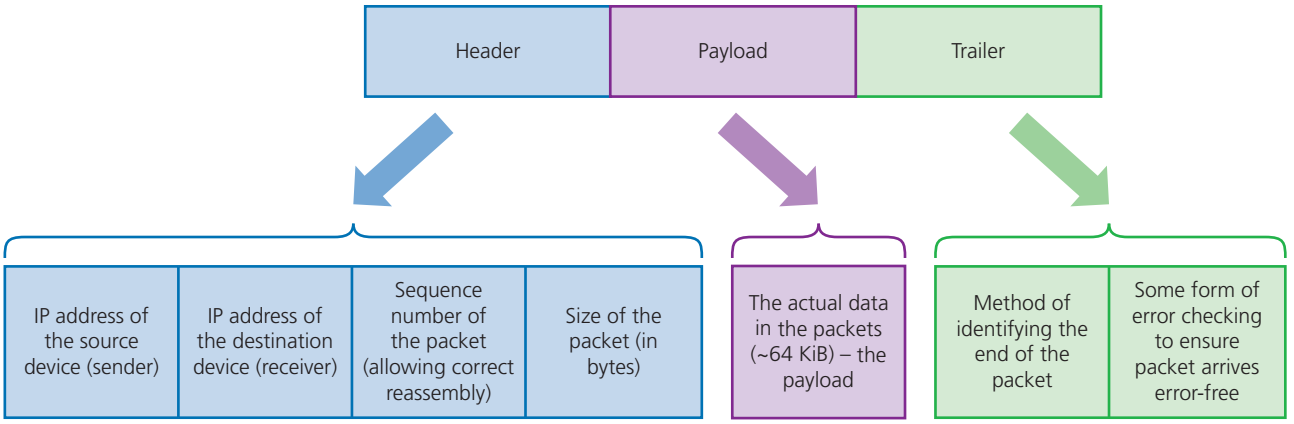
Data sent over long distances is usually broken up into **data packets** (sometimes called datagrams). The packets of data are usually quite small, typically 64 KiB, which are much easier to control than a long continuous stream of data. The idea of splitting up data in this way means each packet can be sent along a different route to its destination. This would clearly be of great benefit if a particular transmission route was out of action or very busy. The only obvious drawback of

splitting data into packets is the need to reassemble the data when it reaches its destination.

Packet structure

A typical packet is split up into:

- a packet header
- the payload
- a trailer.



▲ **Figure 2.1** Packet structure

For **each** packet, the **packet header** consists of:

- the IP address of the sending device
- the IP address of the receiving device
- the sequence number of the packet (this is to ensure that all the packets can be reassembled into the correct order once they reach the destination)
- packet size (this is to ensure the receiving station can check if all of the packets have arrived intact).

(Note: the header often also contains another value indicating how many packets there are in total for this transmission.)

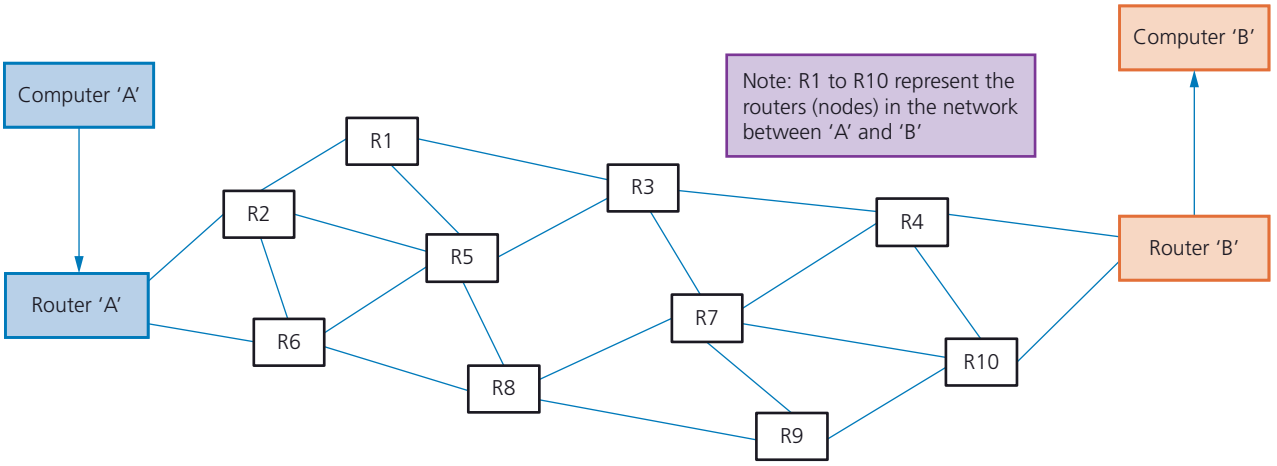
For **each** packet, the **payload** consists of the actual data being sent in the packet (this is usually about 64 KiB).

For **each** packet, the **packet trailer** consists of:

- some way of identifying the end of the packet; this is essential to allow each packet to be separated from each other as they travel from sending to receiving station
- an error checking method; **cyclic redundancy checks** (CRCs) are used to check data packets:
  - this involves the sending computer adding up all the 1-bits in the payload and storing this as a hex value in the trailer before it is sent
  - once the packet arrives, the receiving computer recalculates the number of 1-bits in the payload
  - the computer then checks this value against the one sent in the trailer
  - if the two values match, then no transmission errors have occurred; otherwise the packet needs to be re-sent.

Packet switching

Let us now consider what happens when a photograph, for example, is sent from computer 'A' to computer 'B'. The photograph will be split up into a number of packets before it is sent. There will be several possible routes for the packets, between computer 'A' (sender) and computer 'B' (receiver). Each stage in the route contains a **router**. A router receives a data packet and, based on the information in the header, decides where to send it next. For example:



▲ **Figure 2.2** Typical network showing possible routes between 'A' and 'B'

Packet switching is a method of data transmission in which a message is broken up into a number of packets. **Each** packet can then be sent independently from start point to end point. At the destination, the packets will need to be reassembled into their correct order (using the information sent in the header). At each stage in the transmission, there are **nodes** that contain a router. Each router will determine which route the packet needs to take, in order to reach its destination (the destination IP address is used in this part of the process).

Suppose our photograph (Figure 2.3) has been split up into five packets that have been sent in the following order:



▲ **Figure 2.3**

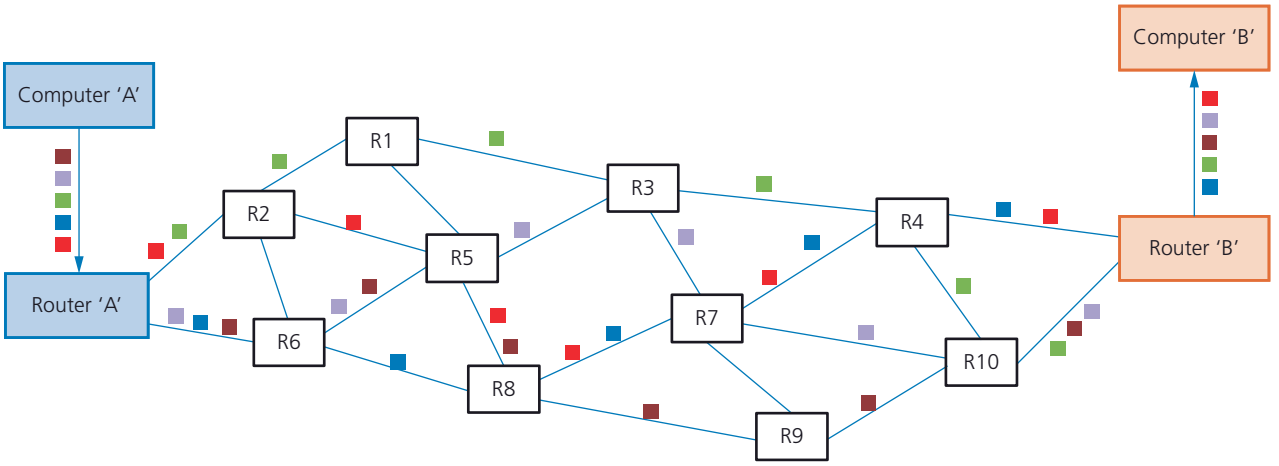
- each packet will follow its own path (route)
- routers will determine the route of each packet
- routing selection depends on the number of packets waiting to be processed at each node
- the shortest possible path **available** is always selected – this may not always be the shortest path that **could** be taken, since certain parts of the route may be too busy or not suitable
- unfortunately, packets can reach the destination in a different order to that in which they were sent.

Figure 2.5 shows one possible scenario. Notice the different paths taken by each packet from computer 'A' to computer 'B'. Also notice that the packets have arrived in a different order compared to the way they were sent, namely:



▲ **Figure 2.4**

Computer 'B' will now have to reassemble the packets into the original sequence.



▲ **Figure 2.5** Typical network showing possible paths taken by each packet

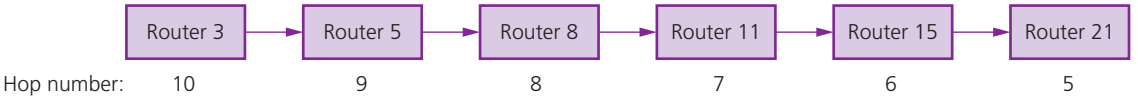
The benefits of packet switching are:

- there is no need to tie up a single communication line
- it is possible to overcome failed, busy or faulty lines by simply re-routing packets
- it is relatively easy to expand package usage
- a high data transmission rate is possible.

The drawbacks of packet switching include:

- packets can be lost and need to be re-sent
- the method is more prone to errors with **real-time streaming** (for example, a live sporting event being transmitted over the internet)
- there is a delay at the destination whilst the packets are being re-ordered.

Sometimes it is possible for packets to get lost because they keep 'bouncing' around from router to router and never actually reach their destination. Eventually the network would just grind to a halt as the number of **lost packets** mount up, clogging up the system. To overcome this, a method called **hopping** is used. A **hop number** is added to the header of each packet, and this number is reduced by 1 *every time it leaves a router* (Figure 2.6).



▲ **Figure 2.6** Hop numbers between routers

Each packet has a maximum hop number to start with. Once a hop number reaches zero, and the packet hasn't reached its destination, then the packet is deleted when it reaches the next router. The missing packets will then be flagged by the receiving computer and a request to re-send these packets will be made.

Find out more

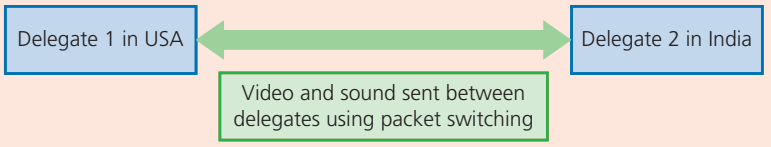
Another method of sending packets is called circuit switching. Find out how this differs to packet switching, and then re-draw Figure 2.5 showing the route the packets take when using circuit switching.

Advice

Hopping is not included on the syllabus but is included here for completeness (to help understand how packets can get lost).

Activity 2.1

1 Suppose a video conference is taking place between delegates in two different countries. Packet switching is being used to send video and sound data between the delegates:



▲ **Figure 2.7**

Describe:

i any potential problems with sound and video quality

ii how these problems could be caused.

2 Explain how packet switching could be used to download a large web page from a website.

3 a The trailer in a packet will use one form of error checking.

Explain what is meant by a cyclic redundancy check.

b The payload contains the following data:

```
11110000 10000011 00110011 00111111 11111110 11100011
```

Use this data to show how the receiving computer can verify that the received payload was error-free.

4 a Explain how it is possible for packets to be lost during their transmission across a network.

b Describe how it is possible for a system to deal with lost packets and prevent them from slowing down the transmission process.

c Explain why you think packet switching might improve data security.

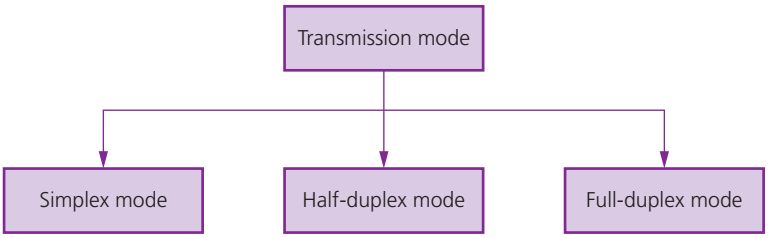
2.1.2 Data transmission

Data transmission can be either over a short distance (for example, computer to printer) or over longer distances (for example, from one computer to another in a global network). Essentially, three factors need to be considered when transmitting data:

- the direction of data transmission (for example, can data transmit in one direction only, or in both directions)
- the method of transmission (for example, how many bits can be sent at the same time)
- how will data be synchronised (that is, how to make sure the received data is in the correct order).

These factors are usually considered by a communication protocol.

Simplex, half-duplex and full-duplex



▲ **Figure 2.8** Transmission modes

Simplex data transmission

Simplex mode occurs when data can be sent in **ONE DIRECTION ONLY** (for example, from sender to receiver). An example of this would be sending data from a computer to a printer.

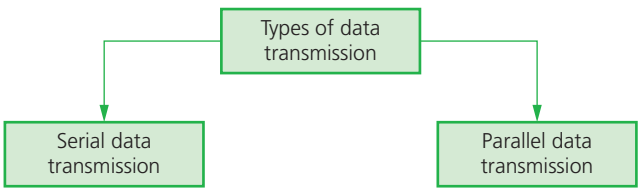
Half-duplex data transmission

Half-duplex mode occurs when data is sent in **BOTH DIRECTIONS** but **NOT AT THE SAME TIME** (for example, data can be sent from 'A' to 'B' and from 'B' to 'A' along the same transmission line, but they can't both be done at the same time). An example of this would be a walkie-talkie where a message can be sent in one direction only at a time; but messages can be both received and sent.

Full-duplex data transmission

Full-duplex mode occurs when data can be sent in **BOTH DIRECTIONS AT THE SAME TIME** (for example, data can be sent from 'A' to 'B' and from 'B' to 'A' along the same transmission line simultaneously). An example of this would be a broadband internet connection.

Serial and parallel data transmission



▲ **Figure 2.9** Types of data transmission

Serial data transmission occurs when data is sent **ONE BIT AT A TIME** over a **SINGLE WIRE/CHANNEL**. Bits are sent one after the other as a single stream.

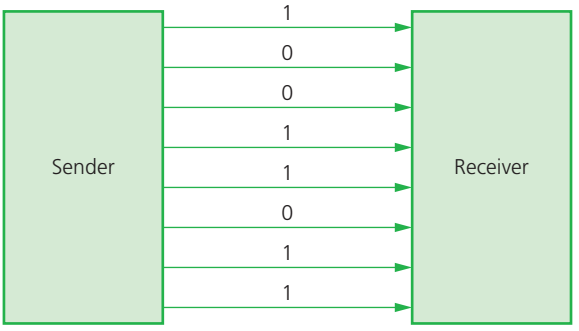


▲ **Figure 2.10** Serial data transmission

(Note: Serial data transmission can be simplex, half-duplex or full-duplex.)

Serial data transmission works well over long distances. However, the data is transmitted at a slower rate than parallel data transmission. Because only one channel/wire is used, data will arrive at its destination fully synchronised (i.e. in the correct order). An example of its use is when connecting a computer to a printer via a USB connection (see Section 2.1.3).

Parallel data transmission occurs when **SEVERAL BITS OF DATA** (usually one byte) are sent down **SEVERAL CHANNELS/WIRES** all at the same time. Each channel/wire transmits one bit:



▲ **Figure 2.11** Parallel data transmission

(Note: Parallel data transmission can be simplex, half-duplex or full-duplex.)

Parallel data transmission works well over short distances. Over longer distances (for example, over 20 metres), data can become **skewed** (that is, the data can arrive unsynchronised) and bits can arrive out of order. The longer the wire, the worse this can become. It is, however, a faster method of data transmission than serial. The internal circuits in a computer use parallel data transmission since the distance travelled between components is very short and high-speed transmission is essential.

Link

For more on data transmission within the CPU refer to Chapter 3.

Activity 2.2

1 Explain what is meant by:
   - i serial, half-duplex data transmission
   - ii parallel, full-duplex data transmission
   - iii serial, simplex data transmission.

2 Which types of data transmission are being described:
   - i data is sent one bit at a time in one direction only
   - ii data is being sent 8 bits at a time in one direction only
   - iii data is being sent 16 bits at a time in both directions simultaneously
   - iv data is sent one bit at a time in both directions simultaneously
   - v data is sent 16 bits at a time in one direction only?

Table 2.1 shows the comparison between serial and parallel data transmission.

▼ **Table 2.1** Comparison of serial and parallel data transmission methods

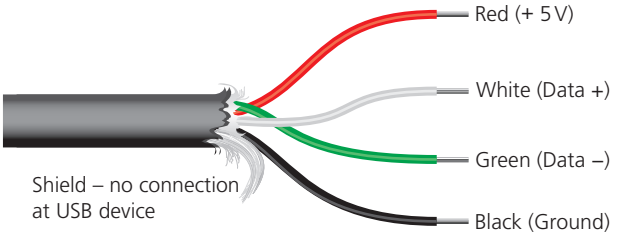
Serial	Parallel
less risk of external interference than with parallel (due to fewer wires)	faster rate of data transmission than serial
more reliable transmission over longer distances	works well over shorter distances (for example, used in internal pathways on computer circuit boards)
transmitted bits won't have the risk of being skewed (that is, out of synchronisation)	since several channels/wires used to transmit data, the bits can arrive out of synchronisation (skewed)
used if the amount of data being sent is relatively small since transmission rate is slower than parallel (for example, USB uses this method of data transmission)	preferred method when speed is important
used to send data over long distances (for example, telephone lines)	if data is time-sensitive, parallel is the most appropriate transmission method
less expensive than parallel due to fewer hardware requirements	parallel ports require more hardware, making them more expensive to implement than serial ports
	easier to program input/output operations when parallel used

2.1.3 Universal serial bus (USB)

Link

Also refer to Section 4.1 on software drivers regarding devices plugged into USB ports.

As the name suggests, the **universal serial bus (USB)** is a form of serial data transmission. USB is now the most common type of input/output port found on computers and has led to a standardisation method for the transfer of data between devices and a computer. It is important to note that USB allows both half-duplex and full-duplex data transmission.



▲ **Figure 2.12** Typical USB cable

As Figure 2.12 shows, the USB cable consists of a four-wired shielded cable, with two wires for power (red and black). The other two wires (white and green) are for data transmission. When a device is plugged into a computer using one of the USB ports:

- the computer automatically detects that a device is present (this is due to a small change in the voltage on the data signal wires in the USB cable)
- the device is automatically recognised, and the appropriate device driver software is loaded up so that the computer and device can communicate effectively
- if a new device is detected, the computer will look for the device driver that matches the device; if this is not available, the user is prompted to download the appropriate driver software (some systems do this automatically and the user will see a notice asking for permission to connect to the device website).

We will now consider the benefits and drawbacks of using the USB system:

▼ **Table 2.2** Benefits and drawbacks of USB systems

Benefits	Drawbacks
devices plugged into the computer are automatically detected and device drivers are automatically loaded up	standard USB only supports a maximum cable length of 5 m; beyond that, USB hubs are needed to extend the cable length
connections can only fit one way preventing incorrect connections being made	
it has become an industry standard, which means considerable support is available	even though USB is backward compatible, very early USB standards (V1) may not always be supported by the latest computers
can support different data transmission rates (from 1.5 Mbps to 5 Gbps)	
no need for external power source since cable supplies +5 V power	even the latest version 3 (V3) and version 4 (V4) USB-C systems have a data transfer rate which is slow compared to, for example, Ethernet connections (Note: USB V2 has a maximum data transfer rate of 480 Mbps.)
USB protocol notifies the transmitter to re- transmit data if any errors are detected; this leads to error-free data transmission	
it is relatively easy to add more USB ports if necessary, by using USB hubs	
USB is backward compatible (that is, older versions are still supported)	

A new type of USB connector, referred to as USB-C, is now becoming more common in laptops and tablets/phones. This is a 24-pin symmetrical connector which means it will fit into a USB-C port either way round. It is much smaller and thinner than older USB connectors, offers 100 watt (20 volt) power connectivity, which means full-sized devices can now be charged and it can carry data at 10 gigabits per second (10 Gbps); this means it can now support 4K video delivery.

USB-C is backward compatible (to USB 2.0 and 3.0) provided a suitable adaptor is used, and is expected to become the new industry standard (universal) format.

Activity 2.3

Ten statements are shown on the left in the table. Each of these statements is either True or False. For each statement, tick (✓) the appropriate column to indicate which statements are true and which are false.

Statement	True	False
Packets have a header which contains the IP address of the sender and the receiver		
Packets don't require any form of error checking		
USBs use a protocol that allows for error-free data transmission between device and computer		
Serial data transmission suffers from data skewing		
The longest cable length supported by USB is 5 metres or less		
Simplex data transmission occurs when data is transmitted one bit at a time		
Full-duplex data transmission involves sending 8 bits of data at a time		
USB uses serial data transfer		
Packet switching prevents loss of any data packets		
USB connections can transfer data using half-duplex or full-duplex		

2.2 Methods of error detection

2.2.1 The need to check for errors

When data is transmitted, there is always a risk that it may be corrupted, lost or even gained.

Errors can occur during data transmission due to:

- interference (all types of cable can suffer from electrical interference, which can cause data to be corrupted or even lost)
- problems during packet switching (this can lead to data loss – or it is even possible to gain data!)
- skewing of data (this occurs during parallel data transmission and can cause data corruption if the bits arrive out of synchronisation).

Checking for errors is important since computers are unable to understand text, for example, if the words are not recognised by its built-in dictionary. Look at the following example of some corrupted text:

Can you raed tihs?

"I cnduo't bvleiee taht I culod aulaclty uesdtannrd waht I was rdnaieg. Unisg the icndeblire pweor of the hmuan mnid, aocdcrnig to rseecrah at Cmabridge Uinervtisy, it dseno't mttaer in waht oderr the lterets in a wrod are, the olny irpoamtnt tihng is taht the frsit and lsat ltteer be in the rhgit pclae. The rset can be a taotl mses and you can sitll raed it whoutit a pboerlm.

Tihs is bucseae the huamn mnid deos not raed ervey ltteer by istlef, but the wrod as a wlohe.

Aaznmig, huh? Yeah and I awlyas tghhuot slelinpg was ipmorantt! See if yuor fdreins can raed tihs too"

(from an unknown source at Cambridge University)

▲ **Figure 2.13** Example of data corruption on a message

Whilst you probably had little problem understanding this text, a computer would be unable to make any sense of it. Data corruption is therefore a very real problem to a computer. Figure 2.13 could be the result of some data corruption following transmission which would make the text unintelligible to a computer. This is why error checking is such an important part of computer technology. The following section considers a number of ways that can be used to check for errors, so that you don't end up with text as shown in Figure 2.13 above!

There are a number of ways data can be checked for errors following transmission:

- parity checks
- checksum
- echo check.

2.2.2 Parity checks, checksum and echo checks

Parity checks

Parity checking is one method used to check whether data has been changed or corrupted following data transmission. This method is based on the number of 1-bits in a byte of data.

The parity can be either called **EVEN** (that is, an even number of 1-bits in the byte) or **ODD** (that is, an odd number of 1-bits in the byte). One of the bits in the byte (usually the most significant bit or left-most bit) is reserved for a **parity bit**. The parity bit is set according to whether the parity being used is even or odd. For example, consider the byte:

	1	1	0	1	1	0	0

parity bit

Advice

Note for the 7 bit number 1110000, the even parity bit would be 1 and the odd parity bit would be 0. The parity bit can be set as a 1 or a 0 for either choice of parity – it just depends on how many 1s are in the byte.

In this example, if the byte is using even parity, then the parity bit needs to be set to 0, since there is already an even number of 1-bits in the byte (four 1-bits). We thus get:

0	1	1	0	1	1	0	0

parity bit

In this example, if the byte is using odd parity, then the parity bit needs to be set to 1, since we need to have an odd number of 1-bits in the byte. We thus get:

1	1	1	0	1	1	0	0

parity bit

Before data is transferred, an agreement is made between sender and receiver regarding which type of parity is being used. Parity checks are therefore being used as a type of transmission protocol.

Activity 2.4

Find the parity bits for each of the following bytes:

	1	1	0	1	1	0	1	even parity being used
	0	0	0	1	1	1	1	even parity being used
	0	1	1	1	0	0	0	even parity being used
	1	1	1	0	1	0	0	odd parity being used
	1	0	1	1	0	1	1	odd parity being used
	1	1	1	1	1	1	0	even parity being used
	1	1	1	1	1	1	0	odd parity being used
	1	1	0	1	0	0	0	odd parity being used
	0	0	0	0	1	1	1	even parity being used
	1	1	1	1	1	1	1	odd parity being used

If a byte has been transmitted from 'A' to 'B', and if even parity is used, an error would be flagged if the byte now had an odd number of 1-bits at the receiver's end. For example (assuming even parity is being used):

byte being sent:	0	1	0	1	1	1	0	0
	parity bit							
byte being received:	0	1	0	0	1	1	0	0
	parity bit							

In this case, the byte received has three 1-bits, which means it now has odd parity; while the sender's byte was using even parity (four 1-bits). This means an error has occurred during the transmission of the byte. The error is detected by the recipient's computer re-calculating the parity of the byte sent. If even parity had been agreed between sender and receiver, then a change in parity in the received byte indicates that a transmission error has occurred.

Activity 2.5

1 Which of the following received bytes indicate an error has occurred following data transmission?

1	1	1	0	1	1	0	1	even parity being used
0	1	0	0	1	1	1	1	even parity being used
0	0	1	1	1	0	0	0	even parity being used
1	1	1	1	0	1	0	0	odd parity being used
1	1	0	1	1	0	1	1	odd parity being used
1	1	1	1	1	1	1	1	odd parity being used
0	0	0	0	0	0	0	0	even parity being used
1	1	1	0	0	0	0	0	odd parity being used
0	1	0	1	0	1	0	1	even parity being used
1	1	1	0	0	0	1	1	odd parity being used

2 In each case, in **question 1**, where an error occurred, can you work out which bit in the byte was changed during data transmission?

If two of the bits change value following data transmission, it may be impossible to locate the error using parity checking.

Let us imagine we are transmitting the following byte, using even parity:

0	**1**	**0**	**1**	**1**	**1**	**0**	**0**

Suppose more than one bit has been modified during data transmission. This means the byte could have reached the destination as any of the following:

0	**1**	**1**	**1**	**1**	**1**	**0**	**1**	six 1-bits

0	1	0	1	0	0	0	0	two 1-bits

0	1	0	1	0	1	1	0	four 1-bits

In all these cases, the byte has clearly been corrupted, but the bytes have retained even parity. Therefore, no error would be flagged in spite of the obvious errors in transmission. Clearly it will be necessary to have other ways to complement parity when it comes to error checking to ensure errors are never missed. One such method is called checksum – see the next section.

You should have concluded that *any* of the bits in question 2 (Activity 2.5) could have been changed where there was a transmission error. Therefore, even though an error has been flagged, it is impossible to know **exactly** which bit is in error.

One of the ways round this problem is to use **parity blocks**. In this method, a block of data is sent and the number of 1-bits are totalled horizontally and vertically (in other words, a parity check is done in both horizontal and vertical directions). As the following example shows, this method not only identifies that an error has occurred but also indicates where the error is.

Example 1

In this example, nine bytes of data have been transmitted. Agreement has been made that **even parity** will be used. Another byte, known as the **parity byte**, has also been sent. This byte consists entirely of the parity bits produced by the vertical parity check. The parity byte also indicates the end of the block of data.

Table 2.3 shows how the data arrived at the receiving end. It is now necessary to check the parity of each byte horizontally (bytes 1 to 9) and vertically (columns 1 to 8). Each row and column where the parity has changed from even to odd should be flagged:

▼ **Table 2.3** Parity block showing nine bytes and parity byte

	Parity bit	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7	Bit 8
Byte 1	1	1	1	1	0	1	1	0
Byte 2	1	0	0	1	0	1	0	1
Byte 3	0	1	1	1	1	1	1	0
Byte 4	1	0	0	0	0	0	1	0
Byte 5	0	1	1	0	1	0	0	1
Byte 6	1	0	0	0	1	0	0	0
Byte 7	1	0	1	0	1	1	1	1
Byte 8	0	0	0	1	1	0	1	0
Byte 9	0	0	0	1	0	0	1	0
Parity byte	1	1	0	1	0	0	0	1

A careful study of Table 2.3 shows the following:

- byte 8 (row 8) now has incorrect parity (there are three 1-bits)
- bit 5 (column 5) also now has incorrect parity (there are five 1-bits).

First of all, the table shows that an error has occurred following data transmission (there has been a change in parity in one of the bytes).

Secondly, at the intersection of row 8 and column 5, the position of the incorrect bit value (which caused the error) can be found. The 1-bit at this intersection should be a 0-bit; this means that byte 8 should have been:

0	0	0	1	0	0	1	0

which would also correct column 5 giving an even vertical parity (now has four 1-bits).

This byte could therefore be corrected automatically as shown above, or an error message could be relayed back to the sender asking them to re-transmit the block of data.

Activity 2.6

1 The following block of data was received after transmission from a remote computer; **odd parity** was being used by both sender and receiver. One of the bits has been changed during the transmission stage. Locate where this error is and suggest a corrected byte value:

	Parity bit	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7	Bit 8
Byte 1	0	1	1	0	0	0	1	0
Byte 2	1	0	1	1	1	1	1	1
Byte 3	1	0	0	1	1	0	0	0
Byte 4	0	1	1	0	1	0	1	0
Byte 5	1	1	1	0	0	1	1	0
Byte 6	1	0	0	0	0	1	0	1
Byte 7	0	1	1	1	0	0	0	0
Byte 8	0	0	0	0	0	0	0	1
Byte 9	0	1	1	1	1	0	1	0
Parity byte	1	0	1	1	1	1	0	0

2 The following block of data was received after transmission from a remote computer. Even parity was being used by both sender and receiver. One of the bytes has been changed during the transmission stage. Locate where this error is and suggest a corrected byte value.

	Parity bit	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7	Bit 8
Byte 1	1	1	0	0	0	0	0	0
Byte 2	0	0	1	1	1	1	0	0
Byte 3	0	1	0	0	0	1	1	1
Byte 4	1	0	1	0	1	1	1	1
Byte 5	0	0	0	1	0	0	0	1
Byte 6	0	0	1	1	1	1	1	1
Byte 7	1	0	1	1	0	1	0	0
Byte 8	0	1	0	1	0	0	0	1
Byte 9	1	1	1	0	0	1	0	0
Parity byte	0	0	1	1	1	0	0	1

Checksum

A **checksum** is a method used to check if data has been changed or corrupted following data transmission. Data is sent in blocks, and an additional value, called the checksum, is sent at the end of the block of data.

The checksum process is as follows:

- when a block of data is about to be transmitted, the checksum is calculated from the block of data
- the calculation is done using an agreed algorithm (this algorithm has been agreed by sender and receiver)
- the checksum is then transmitted with the block of data
- at the receiving end, the checksum is recalculated by the computer using the block of data (the agreed algorithm is used to find the checksum)
- the re-calculated checksum is then compared to the checksum sent with the data block
- if the two checksums are the same, then no transmission errors have occurred; otherwise a request is made to re-send the block of data.

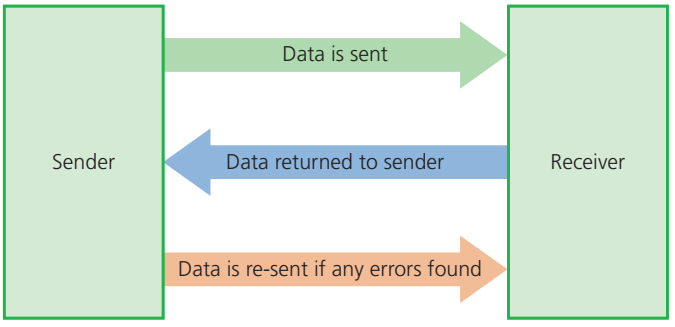
Echo check

With **echo check**, when data is sent to another device, this data is sent back again to the sender. The sender's computer compares the two sets of data to check if any errors occurred during the transmission process.

As you will have no doubt worked out, this isn't very reliable. If the two sets of data are different, it isn't known whether the error occurred when sending the data in the first place, or if the error occurred when sending the data back for checking.

However, if no errors occurred, then it is another way to check that the data was transmitted correctly. In summary:

- a copy of the data is sent back to the sender
- the returned data is compared with the original data by the sender's computer
- if there are no differences, then the data was sent without error
- if the two sets of data are different, then an error occurred at some stage during the data transmission.



▲ **Figure 2.14** Echo check diagram

2.2.3 Check digits

A **check digit** is the final digit included in a code; it is calculated from all the other digits in the code. Check digits are used for barcodes on products, such as International Standard Book Numbers (ISBN) and Vehicle Identification Numbers

(VIN). Check digits are used to identify errors in *data entry* caused by mis-typing or mis-scanning a barcode. They can usually detect the following types of error:

- an incorrect digit entered, for example 5327 entered instead of 5307
- transposition errors where two numbers have changed order, for example 5037 instead of 5307
- omitted or extra digits, for example 537 instead of 5307 or 53107 instead of 5307
- phonetic errors, for example 13 (thirteen), instead of 30 (thirty).



▲ **Figure 2.15** Sample barcode (ISBN 13 code with check digit)

There are a number of different methods used to generate a check digit. Two common methods will be considered here:

- ISBN 13
- Modulo-11

Example 1: ISBN 13

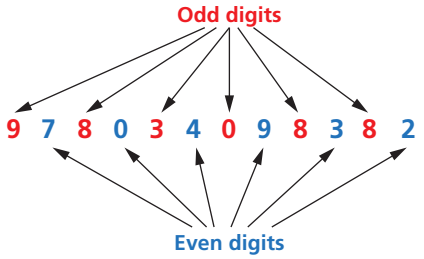
The check digit in ISBN 13 is the thirteenth digit in the number. We will now consider two different calculations. The first calculation is the generation of the check digit. The second calculation is a verification of the check digit (that is, a recalculation).

Calculation 1 – Generation of the check digit from the other 12 digits in a number

The following algorithm generates the check digit from the 12 other digits:

1. add all the odd numbered digits together
2. add all the even numbered digits together and multiply the result by 3
3. add the results from 1 and 2 together and divide by 10
4. take the remainder, if it is zero then use this value, otherwise subtract the remainder from 10 to find the check digit.

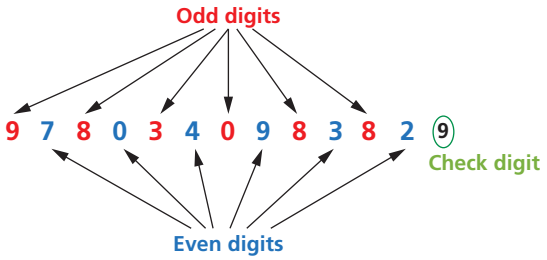
Using the ISBN **9 7 8 0 3 4 0 9 8 3 8 2** (note this is the same ISBN as in Figure 2.15):



▲ **Figure 2.16** ISBN (no check digit)

1. **9 + 8 + 3 + 0 + 8 + 8 = 36**
2. **3 × (7 + 0 + 4 + 9 + 3 + 2) = 75**
3. (**36** + **75**)/10 = 111/10 = 11 remainder 1
4. 10 – 1 = **9** the check digit

So we end up with the following thirteen-digit number (which matches the number shown in Figure 2.15):



▲ **Figure 2.17** ISBN (including the check digit)

Calculation 2 – Re-calculation of the check digit from the thirteen-digit number (which now includes the check digit)

To check that an ISBN 13-digit code is correct, including its check digit, a similar process is followed:

1 add all the odd numbered digits together, **including** the check digit
2 add all the even number of digits together and multiply the result by 3
3 add the results from 1 and 2 together and divide by 10
4 the number is correct if the remainder is zero.

Using the ISBN **9 7 8 0 3 4 0 9 8 3 8 2 9** (including its check digit) from Figure 2.17:

1 **9 + 8 + 3 + 0 + 8 + 8 + 9 = 45**
2 **3 × (7 + 0 + 4 + 9 + 3 + 2) = 75**
3 (45 + 75)/10 = 120/10 = 12 remainder **0**
4 remainder is 0, therefore number is correct.

Advice

You will not need to remember the steps shown in these algorithms; the steps will be given to you, but it is important that you understand how to use an algorithm to calculate or verify check digits.

? Example 2: Modulo-11

The modulo-11 method can have varying lengths of number which makes it suitable for many applications, such as product codes or VINs. The first calculation is the generation of the check digit. The second calculation is a verification of the check digit (that is, a recalculation).

Calculation 1 – Generation of the check digit from the other digits in a number

(In this example, we will assume the original number contained only 7 digits.)

The following algorithm generates the check digit from the other 7 digits:

1 each digit in the number is given a weighting of 8, 7, 6, 5, 4, 3 or 2 starting from the left (weightings start from 8 since the number will become eight-digit when the check digit is added)
2 the digit is multiplied by its weighting and then each value is added to make a total
3 the total is divided by 11
4 the remainder is then subtracted from 11 to find the check digit (note if the remainder is 10 then the check digit 'X' is used).

The example to be used has the following seven-digit number:

1 7-digit number: 4 1 5 6 7 1 0

weighting values: 8 7 6 5 4 3 2

2 sum: (8 × 4) + (7 × 1) + (6 × 5) + (5 × 6) + (4 × 7) + (3 × 1) + (2 × 0)
= 32 + 7 + 30 + 30 + 28 + 3 + 0
total = 130
3 divide total by 11: 130/11 = 11 remainder 9
4 subtract remainder from 11: 11 – 9 = 2 (check digit)

So we end up with the following eight-digit: 4 1 5 6 7 1 0 2

Calculation 2 – Re-calculation of the check digit from the eight-digit number (which now includes the check digit)

To check that the eight-digit number is correct, including its check digit, a similar process is followed:

1 each digit in the number is given a weighting of 8, 7, 6, 5, 4, 3, 2 or 1 starting from the left
2 the digit is multiplied by its weighting and then each value is added to make a total
3 the total is divided by 11
4 the number is correct if the remainder is zero

Using the 8-digit number: 4 1 5 6 7 1 0 2
1 weighting values: 8 7 6 5 4 3 2 1
2 sum: (8 × 4) + (7 × 1) + (6 × 5) + (5 × 6) + (4 × 7) + (3 × 1) + (2 × 0) + (1 × 2)
= 32 + 7 + 30 + 30 + 28 + 3 + 0 + 2
total = 132
3 divide total by 11: 132/11 = 12 remainder 0
4 remainder is 0, therefore number is correct

Activity 2.7

1 Using the algorithm for ISBN-13 calculate the check digit for:

`978151045759`

2 Find the check digits for the following numbers using **both** modulo-11 and ISBN 13 methods:
i 2 1 3 1 1 1 0 0 0 4 2 8
ii 9 0 9 8 1 2 1 2 3 5 4 4

2.2.4 Automatic Repeat Requests (ARQs)

We have already considered parity checks and echo checks as methods to verify that data has arrived at its destination unchanged. An **Automatic Repeat Request (ARQ)** is a third way used to check data following data transmission. This method can best be summarised as follows:

- ARQ uses positive and negative **acknowledgements** (messages sent to the receiver indicating that data has/has not been received correctly) and **timeout** (this is the time interval allowed to elapse before an acknowledgement is received)
- the receiving device receives an error detection code as part of the data transmission (this is typically a Cyclic Redundancy Check – refer to Section 2.1.1); this is used to detect whether the received data contains any transmission errors
- if no error is detected, a positive acknowledgement is sent back to the sending device
- however, if an error is detected, the receiving device now sends a negative acknowledgement to the sending device and requests re-transmission of the data
- a time-out is used by the sending device by waiting a pre-determined amount of time ….
- … and if no acknowledgement of any type has been received by the sending device within this time limit, it automatically re-sends the data until a positive acknowledgement is received ….
- … or until a pre-determined number of re-transmissions has taken place
- ARQ is often used by mobile phone networks to guarantee data integrity.

2.3 Symmetric and asymmetric encryption

2.3.1 The purpose of encryption

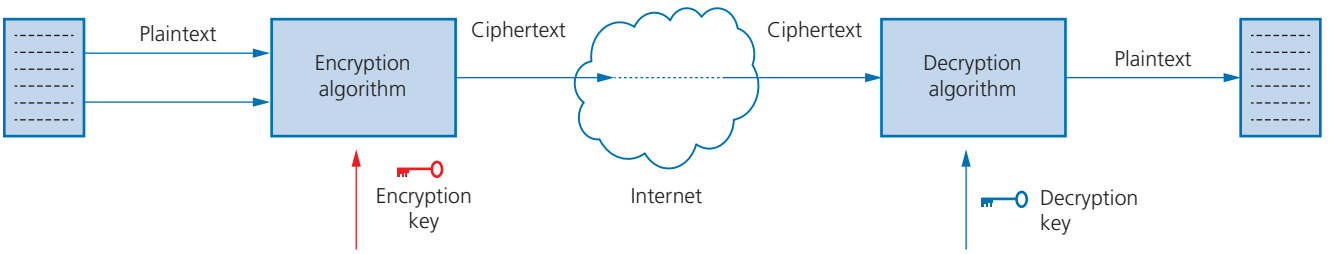
When data is transmitted over any public network (wired or wireless), there is always a risk of it being intercepted by, for example, a hacker. Under these circumstances, a hacker is often referred to as an **eavesdropper**. Using **encryption** helps to minimise this risk.

> **Link**
>
> For more on cyber security see Chapter 5.

Encryption alters data into a form that is unreadable by anybody for whom the data is not intended. It cannot prevent the data being intercepted, but it stops it from making any sense to the eavesdropper. This is particularly important if the data is sensitive or confidential (for example, credit card/bank details, medical history or legal documents).

Plaintext and ciphertext

The original data being sent is known as **plaintext**. Once it has gone through an **encryption algorithm**, it produces **ciphertext**:



▲ **Figure 2.18** Plaintext and ciphertext

2.3.2 Symmetric and asymmetric encryption

Symmetric encryption

Symmetric encryption uses an encryption key; the same key is used to encrypt and decrypt the encoded message. First of all, consider a simple system that uses a 10-digit denary encryption key (this gives 1×10^{10} possible codes); and a decryption key. Suppose our encryption key is:

4 2 9 1 3 6 2 8 5 6

which means every letter in a word is shifted across the alphabet +4, +2, +9, +1, and so on, places. For example, here is the message COMPUTER SCIENCE IS EXCITING (plaintext on the top line of Figure 2.19) before and after applying the encryption key (forming the ciphertext shown on the bottom line of Figure 2.19):

C	O	M	P	U	T	E	R
4	2	9	1	3	6	2	8
G	Q	V	Q	X	Z	G	Z

S	C	I	E	N	C	E
5	6	4	2	9	1	3
X	I	M	G	W	D	H

I	S
6	2
O	U

E	X	C	I	T	I	N	G
8	5	6	4	2	9	1	3
M	C	I	M	V	R	O	J

▲ **Figure 2.19** Plaintext into ciphertext using 10-digit encryption key

To get back to the original message, it will be necessary to apply the same decryption key; that is, 4 2 9 1 3 6 2 8 5 6. But in this case, the decryption process would be the reverse of encryption and each letter would be shifted –4, –2, –9, –1, and so on. For example, 'G' → 'C', 'Q' → 'O', 'V' → 'M', 'Q' → 'P', and so on.

However, modern computers could 'crack' this encryption key in a matter of seconds. To try to combat this, we now use 256-bit binary encryption keys that give 2^{256} (approximately, 1.2×10^{77}) possible combinations. (Even this may not be enough as we head towards **quantum computers**.)

Find out more

One of the ways of mitigating the risk of symmetric keys falling into the wrong hands (known as the key distribution problem) is to use a system based on modulo-11, where both sender and receiver can calculate the encryption key without it actually being exchanged in any way. Find out how this system works.

The real difficulty is keeping the encryption key a secret (for example, it needs to be sent in an email or a text message which can be intercepted). Therefore, the issue of security is always the main drawback of symmetrical encryption, since a single encryption key is required for both sender and recipient.

Asymmetric encryption

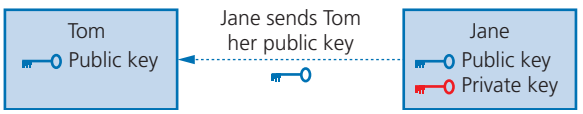
Asymmetric encryption was developed to overcome the security problems associated with symmetric encryption. It makes use of two keys called the **public key** and the **private key**:

- public key (made available to everybody)
- private key (only known to the computer user).

Both types of key are needed to encrypt and decrypt messages.

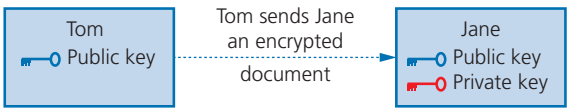
We will use an example to explain how this works; suppose Tom and Jane work for the same company and Tom wishes to send a confidential document to Jane:

1. Jane uses an algorithm to generate a **matching pair of keys** (private and public) that they must keep stored on their computers; the matching pairs of keys are mathematically linked but can't be derived from each other.
2. Jane now sends her public key to Tom.



▲ **Figure 2.20** Jane sends Tom her public key

3. Tom now uses Jane's public key (🔑) to encrypt the document he wishes to send to her. He then sends his encrypted document (ciphertext) back to Jane.



▲ **Figure 2.21** Encrypted document sent from Tom to Jane

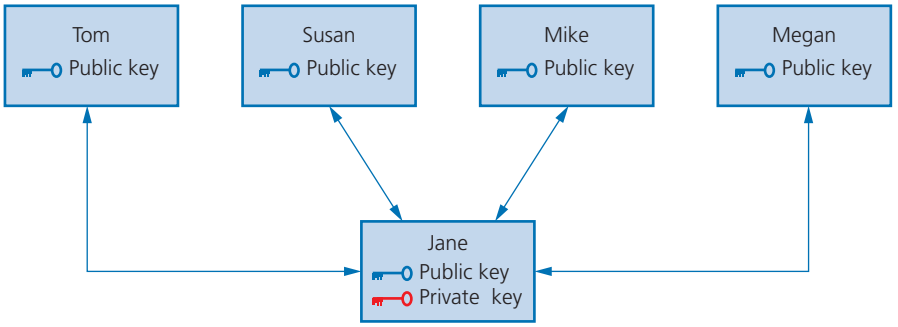
4. Jane uses her **matching** private key (🔑) to unlock Tom's document and decrypt it; this works because the public key used to encrypt the document and the private key used to decrypt it are a matching pair generated on Jane's computer. (Jane can't use the public key to decrypt the message.)

Activity 2.8

1 At the moment Jane can only receive encrypted documents from Tom.
Describe what would need to happen for Jane to be able to **send** encrypted documents back to Tom.

2 Explain why this method is much more secure than symmetric encryption.

Jane can also exchange her public key with any number of people working in the company, so she is able to receive encrypted messages (which have been encrypted using her public key) and she can then decrypt them using her matching private key:



▲ **Figure 2.22** The sharing of Jane's public key

However, if a two-way communication is required between all five workers, then they all need to generate their own matching public and private keys. Once this is done, all users then need to swap public keys so that they can send encrypted documents/files/messages between each other. Each worker will then use their own private key to decrypt information being sent to them.

Find out more

1 Using Figure 2.22 as your template, draw a new diagram showing the public keys and private keys that need to be swapped if Jane wishes to have a two-way exchange of encrypted documents between Tom, Susan, Mike and Megan.

2 Consider the complexity, if all five people want to have secure two-way communication between each other (and not just with Jane). This would mean each of the five workers sharing their own public keys with each of the other workers.

Activity 2.9

For each of the following ten questions, choose which of the five options corresponds to the correct response.

a What is meant by the term **ciphertext** when used in encryption?

A an encryption or decryption algorithm

B an encrypted message

C a type of session key

D another name for plaintext

E text following an encryption algorithm

b When carrying out asymmetric encryption, which of the following users would keep the private key?

A the sender
B the receiver
C both sender and receiver
D all recipients of the message
E none of the above

c In encryption, which of the following is the term used to describe the message before it is encrypted?

A simpletext
B plaintext
C notext
D ciphertext
E firsttext

d Which of the following is the biggest disadvantage of using symmetric encryption?

A it is very complex and time consuming
B it is rarely used anymore
C the value of the key reads the same in both directions
D it only works on computers with older operating systems
E there is a security problem when transmitting the encryption key

e Which of the following is the correct name for a form of encryption in which both the sender and the recipient use the same key to encrypt and decrypt?

A symmetric key encryption
B asymmetric key encryption
C public key encryption
D same key encryption
E block cipher encryption

f What of the following is the final number in a code, which is calculated from all the numbers in the code; its purpose is to find errors in data entry?

A parity check
B checksum
C cyclic redundancy check
D parity bit
E check digit

g Which of the following is a form of error detection that makes use of a system of acknowledgements and timeouts?

A automatic repeat request
B echo check
C check digit
D parity bit
E cyclic redundancy check

h Which of the following methods uses an extra bit added to a byte to ensure it contains an even number of 1s or odd number of 1s?

A cyclic redundancy check
B parity check
C checksum
D check digit
E echo check

i Which of the following uses a calculated value which is sent after a block of data; the receiving computer also calculates the value from the block of data and compares the values?

A parity check
B check digit
C packet switching
D checksum
E automatic repeat request

j Which of the following describes the check where the receiving computer sends back a copy of the data to the sending computer to allow it to compare the data?

A echo check
B automatic repeat request
C checksum
D parity check
E check digit

Extension

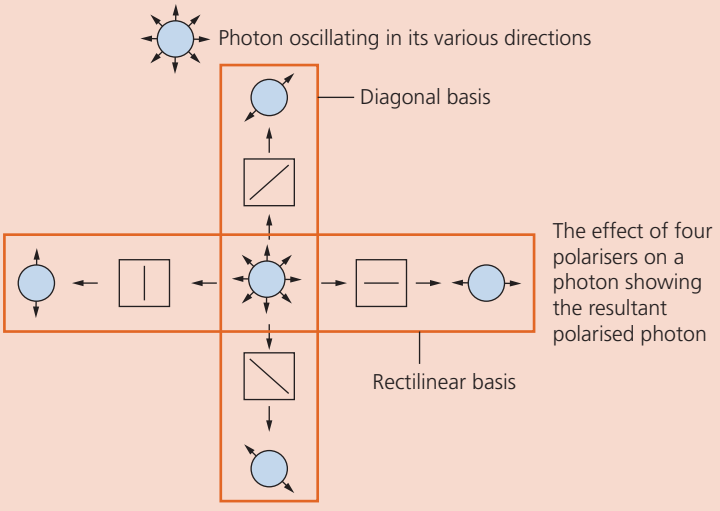
For those students considering the study of this subject at A Level, the following section gives some insight into further study on encryption.

Quantum cryptography exploits the laws of quantum mechanics to improve on the security of data. Quantum cryptography is based on the use of particles of light called photons (with energy calculated according to the formula: E = hf) and their physical quantum properties to produce a virtually unbreakable encryption system. This helps protect the security of data being transmitted over fibre optic cables. The technology is based on the laws of physics, rather than mathematics which is how the current cryptography methods already covered in this chapter work.

One of the uses of quantum cryptography is when sending encryption keys across a network – this uses a **Quantum Key Distribution (QKD)** protocol (one of the most common is called BB84).

QKD uses quantum mechanics to ensure a secure transmission of encryption keys. They use a **Qubit** (**Qu**antum **bit**) which is the basic unit of quantum 'data'. Unlike normal binary (which uses discrete 0s and 1s), the state of a Qubit is **both** 0 and 1 until the photon is measured.

A photon normally vibrates or oscillates in all different directions. Polarisation restricts these vibrations to particular directions. The following diagram shows how a photon can be polarised in one of two **bases** – the rectilinear (up/down and side-to-side) basis and the diagonal basis. Do not worry too much about what a basis is – you can just think of them as two different ways of preparing and measuring the photon.



▲ **Figure 2.23** Quantum cryptography

So how do we use quantum cryptography to send an encryption key from 'A' to 'B' using the QKD protocol? To understand this we need to note that:

- Photons can be polarised in one of two bases – rectilinear or diagonal.
- In the rectilinear basis, 1 can be represented by ↕ and 0 by ↔. In the diagonal basis 1 is ↗ and 0 is ↖.
- A photon that is polarised in the rectilinear basis will always give the same result when measured in the rectilinear basis. So, if a photon is polarised as '1' in the rectilinear basis (↕) it will always be measured in the rectilinear basis to have the polarisation ↕.
- Similarly, a photon that is polarised in the diagonal basis will always give the same result when measured in the diagonal basis. So, if a photon is polarised as '1' in the diagonal basis (↗) it will always be measured in the diagonal basis to have the polarisation ↗.
- However, if a photon is polarised in the rectilinear basis but measured in the diagonal basis then information about the original polarisation is lost. The result of the measurement has a 50-50 chance of being ↗ or ↖ because in the diagonal basis the photon is in both 1 and 0 states **at the same time!** This measurement tells the receiver nothing about the photon's original polarisation in the rectilinear basis.
- Similarly, if a photon is polarised in the diagonal basis but measured in the rectilinear basis then information about the original polarisation is lost. The result of the measurement has a 50-50 chance of being ↕ or ↔ because in the rectilinear basis the photon is in both 1 and 0 states **at the same time**! This tells the receiver nothing about the photon's original polarisation in the diagonal basis.

1. So, the sender polarises each photon using a basis that is selected at random.
2. The receiver measures each photon using a basis that is selected at random.
3. The sender and receiver publicly exchange which basis they used for each photon.

Only when the sender and receiver use the same basis for measurement can they be sure that are both reading a 1 or 0. When they used the same basis the receiver knows they have measured a '1' or '0' correctly.

In this chapter, you have learnt about:

- ✔ the use of data packets when sending data over networks
- ✔ packet switching
- ✔ types of data transmission (serial, parallel, simplex, half-duplex and full-duplex)
- ✔ the universal serial bus (USB)
- ✔ how errors during transmission can occur and how to recognise them and recover from them (using parity check, checksum and echo check)
- ✔ the use of check digits to identify errors following data entry
- ✔ how automatic repeat requests are used to find errors in transmitted data
- ✔ why encryption of data is used
- ✔ symmetric and asymmetric encryption
- ✔ the use of public keys and private keys in asymmetric encryption.

Key terms used throughout this chapter

data packet – a small part of a message/data that is transmitted over a network; after transmission all the data packets are reassembled to form the original message/data

packet header – the part of the data packet that contains the IP addresses of the sender and receiver, and includes the packet number which allows reassembly of the data packets

packet trailer – the part of a data packet that indicates the end of the data packet and cyclic redundancy check error check

cyclic redundancy check (CRC) – an error checking method in which all the 1-bits in the data packet payload are added and the total is stored in the packet trailer; the same calculation is repeated at the receiving station

payload – the actual data being carried in a data packet

node – stages in a network that can receive and transmit data packets; routers are nodes in communication networks

packet switching – a method of transmission in which a message is broken into many data packets which can then be sent along pathways independently of each other

router – a device that enables data packets to be moved between different networks, for example to join a LAN to a WAN

real time streaming – the transmission of data over a network for live events where the data is sent as soon as it is received or generated

hopping/hop number – a number in a data packet header used to stop data packets that never reach their destination from 'clogging up' the data paths/routes

simplex – data that can be sent on one direction only

half-duplex – data that can be sent in both directions but not at the same time

full-duplex – data that can be sent in both directions at the same time (simultaneously)

serial data transmission – sending data down one channel/wire one bit at a time

parallel data transmission – sending data down several channels/wires several bits at a time (usually 1 byte)

skewed (data) – data that arrives at the destination with the bits no longer synchronised

universal serial bus (USB) – a type of serial data transmission which has become the industry standard for connecting computers to devices via a USB port

parity check – a method used to check if data has been transferred correctly; it makes use of even parity (an even number of 1-bits) or odd parity (an odd number of 1-bits)

parity bit – a bit (either 0 or 1) added to a byte of data in the most significant bit position; this ensures that the byte follows the correct even parity or odd parity protocol

parity block – a horizontal and vertical parity check on a block of data being transmitted

parity byte – an extra byte of data sent at the end of a parity block; it is composed of the parity bits generated from a vertical parity check of the data block

checksum – a verification method used to check if data transferred has been altered or corrupted; calculated from the block of data of data being sent; the checksum value is sent after each data block

automatic repeat request (ARQ) – a method of checking transmitted data for errors; it makes use of acknowledgement and timeout to automatically request re-sending of data if the time interval before positive acknowledgement is too long

acknowledgement – a message sent to the receiver indicating that data has been received correctly (used in the ARQ error detection method)

timeout – the time interval allowed to elapse before an acknowledgement is received (in the ARQ error detection method)

echo check – a method used to check if data has been transferred correctly; data is sent to a receiver and then immediately sent back to the sender; the sender then checks if the received data matches the sent data

check digit – an additional digit appended to a number to check if the entered number is error-free; check digit is a data entry check and not a data transmission check

eavesdropper – another name for a hacker who intercepts data being transmitted on a wired or wireless network

encryption – the process of making data meaningless using encryption keys; without the correct decryption key the data cannot be decoded (unscrambled)

plaintext – the original text/message before it is put through an encryption algorithm

ciphertext – encrypted data that is the result of putting a plaintext message through an encryption algorithm

encryption algorithm – a complex piece of software that takes plaintext and generates an encrypted string known as ciphertext

symmetric encryption – a type of encryption in which the same encryption key is used both to encrypt and decrypt a message

asymmetric encryption – a type of encryption that uses public keys and private keys to ensure data is secure

public key – a type of encryption key that is known to all users

private key – a type of encryption key which is known only to the single computer/user

quantum computer – a computer that can perform very fast calculations; it can perform calculations that are based on probability rather than simple 0 or 1 values; this gives a quantum computer the potential to process considerably more data than existing computers

Exam-style questions

1 A company owns a number of vending machines. Data is sent from each of these machines at the end of the day. The data contains amount of money taken, products sold and any error conditions/reports.

a The company uses both echo checking and automatic repeat requests (ARQs).

i Describe how echo checks work. Explain whether this is a suitable error checking method in this application. [2]

ii Describe how automatic repeat request (ARQ) works. [3]

b ***Checksum*** and ***check digit*** are two terms often confused by students.

Describe **three** differences of the two techniques. [3]

2 Explain each of the following computer terms:

i packet switching

ii cyclic redundancy check

iii data skewing

iv universal serial bus

v parity bit. [10]

3 Eight descriptions are given in the following table. The table columns are labelled checksum, parity check and ARQ.
Tick (✓) the appropriate column which correctly matches each description to the error-checking technique. For each description, it is possible to match 1, 2, 3 or none of the error-checking methods. [8]

Description	Checksum ✓	Parity check ✓	ARQ ✓
extra bit sent with each byte of data			
makes use of timeout and acknowledgement			
if an error is found, a request is made to re-send the data			
check on whether a data packet has been changed following transmission			
re-calculation made on any additional data values sent to the recipient			
data is transmitted in blocks or packets			
a method that can determine which bit in a data stream has been changed			
additional value sent at the end of a block of data to be used to check if any data transmission errors occurred			

4 **a** Four statements about automatic repeat requests (ARQs) are given below, but they are not in the correct order. Put the statements into their correct sequence.

i the sending computer waits for a period of time to see if the receiving computer acknowledges receipt of the data

ii after a set time period, a timeout occurs which automatically triggers the re-sending of the data

iii the sending computer transmits a block of data to the receiving computer
iv this continues until the receiving computer sends an acknowledgement that the data has been received [3]

b Five statements about checksum error checking are given below, but they are not in the correct order. Put the statements into their correct sequence.

i if the two checksum values don't match, the receiving computer requests the data to be re-transmitted
ii the sending computer sends a block of data together with the checksum value
iii the receiving computer uses the block of data it receives to re-calculate the checksum using the same method as the sending computer
iv the two checksum values are compared by the receiving computer
v the sending computer uses the block of data to calculate the checksum using an agreed method [4]

c Five statements about parity checking are given below, but they are not in the correct order. Put the statements into their correct sequence.

i the sending computer sends the binary data including the parity bits
ii the sending and receiving computers agree the parity protocol (odd or even)
iii the sending computer adds a parity bit to each byte to make the byte odd or even parity
iv the receiving computer checks the parity of each byte received and checks it against the agreed protocol
v if the parity of the byte is incorrect, the receiving computer requests the data to be re-sent [4]

d Six statements about check digits are given below, but they are not in the correct order. Put the statements into their correct sequence.

i a human operator will be asked by the computer to re-enter the numerical code
ii the computer calculates the check digit based on the numerical code entered into the computer by a human operator
iii if the two check digits don't match, the human operator has made an error when entering the numerical code
iv the computer compares the calculated check digit with the check digit typed in by the human operator
v a human operator types in the numerical code into the computer
vi the check digit is calculated and added to the numerical code [5]

5 a Describe what is meant by ***symmetric encryption***. [2]

b Describe what is meant by ***asymmetric encryption***. [3]

c Explain why encryption is used when transmitting data over a network. [2]

6 Six descriptions are shown on the left and ten computer terms on the right.
By drawing lines, connect each description to the correct computer term (not all of the computer terms will be used). [6]

a method of error detection; a value is calculated from a block of data and is sent with the block of data during data transmission	skewed data
a method of error detection; it is based on counting the number of 1-bits; uses an additional bit which is the most significant bit in the byte	half-duplex
a data transmission method where data can be sent in both directions at the same time (simultaneously)	checksum
a data transmission method where data is sent one bit at a time over a single channel/wire	ARQ
a data error occurring when data arrives at the destination out of synchronisation	full-duplex
a form of serial data transmission which allows devices to communicate with a computer; it has become the industrial standard	check digit
	universal serial bus
	encryption
	serial
	parity check

7 A file server is used as a central data store for a network of computers. Rory sends data from his computer to a file server that is approximately 100 metres away. It is important that the data is transmitted accurately. Rory needs to be able to read data from and write data to the file server at the same time.

a i Use ticks (✓) to identify the most suitable data transmission methods for this application. [2]

Method 1	Tick ✓	**Method 2**	Tick ✓
Serial		Simplex	
Parallel		Half-duplex	
		Duplex	

ii Explain why your answer to **a i** is the most suitable data transmission. [4]

b Identify and describe **two** methods of error checking that can be used to make sure that the data stored after transmission is accurate. [6]

Cambridge O Level Computer Science 2210, Paper 12 Q4, Oct/Nov 2017

8 Maisey purchases a new router and attaches it to her computer.
The connection she sets up uses duplex data transmission.

a **Five** statements are given about duplex data transmission.
Tick (✓) to show if the statement is **True** or **False**. [5]

Statement	True ✓	False ✓
Duplex data transmission can be either serial or parallel		
Duplex data transmission is when data is transmitted both ways, but only one way at a time		
Duplex transmission is always used to connect a device to a computer		
Duplex data transmission is when data is transmitted both ways at the same time		
Duplex data transmission automatically detects any errors in data		

b Maisey's computer uses an integrated circuit (IC) for data transmission that sends multiple bits at the same time.
State whether the IC uses **serial** or **parallel** data transmission. [1]

c Maisey purchases a new printer and connects it to her computer using the USB port.
Explain **two** benefits of using a USB connection. [4]

Cambridge O Level Computer Science 2210, Paper 12 Q9, Oct/Nov 2019